

Projektuppgift

Objektorienterad problemlösning och Design (GIK2XJ)
Högskolan dalarna

Innehåll

Projektexamination.....	2
Formalia gällande examinationen.....	2
Arbetsform.....	2
Hjälpmedel.....	2
Information om plagiat.....	2
Projektstart ochHandledning.....	2
Bokning av möten för projektstart och handledning.....	2
Projektstart.....	2
Handledning.....	3
Projektredovisning.....	3
Inlämning.....	3
Betygsgränser.....	3
Uppgiftsbeskrivning.....	3
Uppgiften.....	4
Uppgiftskrav.....	5
Baserat på vecka 1, Problemlösning – analysera ett problem.....	5
Baserat på vecka 2, APIE.....	5
Baserat på vecka 3, SOLID.....	5
Baserat på vecka 4, Datastrukturer.....	5
Baserat på vecka 5, Design Patterns.....	5
Baserat på vecka 6, Algoritmer.....	6
Demonstrationer.....	6
För betyget G.....	6
För betyget VG.....	6
Bedömning.....	7
Avslutning.....	7

Projektexamination

"Kursens övergripande mål är att den studerande ska tillägna sig grundläggande kunskaper och färdigheter i objektorienterad teknik för att skapa mjukvara utifrån kravspecifikation och notationsspråk som är välstrukturerad, förståelig, förvaltnings- och skalbar."

Formalia gällande examinationen

Arbetet planeras starta i början av vecka 42 (14/10) och avslutas under vecka 45 (6/11).

Arbetsform

Projektet utförs som ett grupparbete där kursens lärare agerar intressenter. Se rubriken Uppgiftbeskrivning för mer information. Varje student i gruppen ska vid slutredovisningen kunna redogöra för *hela* gruppens arbete och resultat.

Hjälpmedel

Man får använda sig av böcker, internet, tidigare arbeten o.s.v. men se upp för plagiering. Plagiering är i kort att lämna in någon annans arbete som sitt eget. Var noga med att ange källor om du väljer att använda dig av texter från en annan källa. Som regel ska du kunna redogöra för all kod du skrivit och lämnat in. Detta kan ibland ske omedvetet och därför är det viktigt att du läser anvisningen under rubriken "information om plagiat" nedan.

För att underlätta arbetsfördelningen i grupparbetet kan man använda en enkel **tidsrapporteringsmall**. Tidsrapportering är frivilligt att använda i gruppen och är något som ej behöver lämnas in.

Information om plagiat

Genom Din aktiva handling att lämna in denna examination, så bekräftar/intygar Du att Du har läst, förstått och följt följande instruktioner

Referera: <https://www.du.se/sv/bibliotek/referera/>

Fusk och disciplinärenden: <https://www.du.se/sv/studentwebb/rattigheter-och-regler/fusk-och-disciplinarenden/>

Projektstart ochHandledning

Bokning av möten för projektstart och handledning

Handledningsanvisningarna finns endast angivna i Canvas > Uppgifter> Projektuppgift

Notera att uppdateringar kan behöva göras under projekttiden.

Projektstart

Senast torsdag 17/10 kl. 12.00 ska alla grupper ha genomfört ca 10 minuter projektstart-check, dvs. gruppen presenterar sitt valda projekt, projektförklaring och projektstatus (utfört arbete hittills). Se Canvas för mer hjälp och anvisningar.

Ansvariga lärare: hjo@du.se och hje@du.se

Handledning

Under främst vecka 43 och 44 (samt en sista minuten tid v45) finns det möjlighet att boka handledning. Under v45 redovisar gruppen sitt arbete. Se Canvas för mer hjälp och anvisningar.

Projektreddovisning

Projektet redovisas via en presentation (ex.vis Powerpoint) och demonstration(er) av vald lösning. Varje grupp har 20 minuter på sig att redovisa. Varje student i gruppen ska kunna redogöra för hela projektarbetet vid presentationen.

Se rubriken Betygsgränser för mer information.

Var noga med checklistan i uppgiftsbeskrivningen, missa inget som är ett krav. Fysisk närvaro är ett krav, det kommer inte finnas Zoom-möjligheter.

Grupperna redovisar parvis (två grupper/timme) varje hel timme, så det alltid finns några åhörare och tid för frågor. Redovisningsschema delges via Canvas under v44. Vi vill först se hur fördelningen av "uppdrag" blivit för att skapa så blandade grupper som möjligt. Mer information kommer ...

Inlämning

Inlämning sker i Canvas kursrum **senast tisdag den 5/11 klockan 16:00**. Inlämningen ska innehålla minst en presentation och även projektet i form av mjukvara, samt bilagor i form av andra dokumentationsformer om ni så önskar.

Betygsgränser

I gruppens inlämnade arbete ska det framgå vilket betyg gruppen avser att **redovisa för (G eller VG)**.

För betyget G

- Projektet är inlämnat i tid (om man inte lämnar in i tid får man inte presentera)
- Projektarbetet har sådan kvalitet att det kan presenteras och förstås av åhörare
- Projektarbetet uppnår vid redovisningen uppgiftskraven för G, se rubrikerna Demonstrationer och Bedömning

För betyget VG

- Allt för G

samt

- Projektarbetet uppnår vid redovisningen uppgiftskraven för VG, se rubrikerna Demonstrationer och Bedömning

Betyget meddelas inte på plats utan det sätts via Canvas. Om det finns behov av kompletteringar delges det via kommentarer i Canvas. Betyget sätts individuellt trots att det är ett grupparbete.

Uppgiftsbeskrivning

Uppgiften (projektarbetet) är ett slags rollspel där gruppen kan liknas vid en IT-konsultfirma och vi lärare kommer spela flera roller i form av intressenter (s.k. Stakeholders). Mest okunniga kunder/beställare, men även lärare när ni behöver *generell* hjälp och ibland som testorakel: <https://www.google.com/search?q=testorakel> som förstår vad ni menar och kan resonera kring tekniska frågor.

Uppgiften

Er grupp är en IT-konsultfirma som tagit åt sig ett uppdrag. Kunden har ett behov i form av att de behöver få en programvara utvecklad. Kunden kan inget om system/mjukvaruutveckling eller IT. De vet vad de vill ha, men inte hur. Ni ska ta er an deras uppdrag, arbeta med det och komma tillbaka med ett förslag på en lösning.

Presentationen i slutet av projektet är då ni ska "sälja" in er lösning. Ni måste förklara hur ni kommit fram till er föreslagna lösning. Ni behöver få kunden att förstå processen ni tagit er igenom och vilka alternativ ni övervägt.

Vägen till ert lösningsförslag är viktig eftersom kunden inte nöjer sig med att höra - "Gör så här ..." etc. De vill veta varför. De kommer satsa stora pengar på sin IT-lösning och behöver därför vara övertygade om att er lösning är genomtänkt och värt den investering det handlar om. De har redan betalat dyrt arvode för er tid, så de vill ha "bevis" för att ni verkligen lagt ned tillräckligt med tankearbete på lösningen.

Kunden känner kanske inte förtroende för säljare som står och läser innantill på något digitalt hjälpmedel. Presentationens punkter ska räcka och man ska vara vänd mot åhörarna.

Ta gärna foton eller skärm-dumpar på delar av er SDLC (Software/System Development Life Cycle) process eller för anteckningar, som en slags dagbok som ni sedan kan hänvisa till under presentationen. En vägg full med post-it lappar? En "whiteboard" fullkladdad med brainstorming? DevSecOps (Github, GitLab, Azure DevOps, ...). Datorer som utför SUT (System Under Test)?

Visa kunden er programutvecklingsprocess (Software development process):

https://en.wikipedia.org/wiki/Software_development_process 😊

Ni ska välja ett av nedanstående uppdrag

1. Biljettshoppen

Kunden vill utveckla en mjukvara för att sälja evenemangsbiljetter. De kommer endast ha ett fast antal biljetter att sälja per evenemang. Biljetterna kommer att släppas vid ett annonserat datum och klockslag. Sittplatserna är antingen av fällstolar eller bänkar. Köparna ska kunna få välja plats, de ska kunna se lediga platser och tagna platser på arenan. Evenemangen kommer vara inomhus. Det ena huset är byggt av tegel medan de andra är av trä. Bokningarna är tidsbestämda så köparna måste göra ett köp inom 10 minuter annars förlorar de sin bokning. De ska kunna betala direkt eller välja på faktura. De ska kunna köpa en eller flera biljetter samtidigt men max 5 per person.

2. Brädspel

Kunden vill ha ett övningsprogram för att träna spelstrategier på något eller några enklare brädspel (Exempel: Othello/Reversi, Checkers eller lämpligt kortspel). I spelet ska användaren och en motståndare utföra sina drag. När man vinner ska en segermelodi spelas upp på det lokala nätverket. Man ska kunna backa ("undo") om man tycker att man kört fast, och göra ett annat drag i stället. En spelare ska inte kunna genomföra ogiltiga drag. Man ska kunna spara ett parti för att senare spela upp det igen, drag för drag. För att bemöta färgblindhet ska spelarna kunna välja färg på plattform, spelsymboler och eventuella tärningar.

3. Personalregister

Kunden vill ha ett digitalt personalregister för sina arbetsmyror med över 500 000 anställda. De behöver kunna lägga till, ta bort, söka och uppdatera personal. Arbetskläderna ingår inte i

tjänsten men de förväntas, minst, ha skyddsskor. Myror har tyvärr kort livsspann och de dör efter två veckor. Men det föds fler varje vecka än det dör. Kunden är rädd för konventionella RDBMS (Relational Database Management System) eftersom databasattacker är vanliga. Kunden vill därför använda någon DBMS variant för att lagra data persistent i någon form. Tex. i RAM-minnet eftersom man har oändlig UPS (Uninterruptible Power Supply) så systemet alltid är i drift efter uppstart. Kunden har också börjat fundera på att anställa arbetsbin men har inte hittat en lämplig drottning än.

4. Lottospel

Kunden har kommit på att de vill konkurrera ut svenska spel med ett eget lotteri. Alla deras spelare kan få skapa en eller flera lottorader med hjälp av 5 numeriska värden. Varje fredag ska en lottorad dras och om det finns vinnare får de 75% av intäkterna från den veckan. Om ingen vinnare finns ackumuleras vinsten till nästkommande vecka. Efter varje dragning ska alla lottorader tömmas och spelarna får köpa en ny lottorad för nästa vecka. Alla intäkter ska sparas till en bank i Schweiz via SEPA (Single Euro Payment Area). En stamkund kan få prenumerera på en lottorad och de får betala lite mindre men måste då vara en registrerad spelare. Kunden vill kunna låta lotteriet växa om fler börjar spela, de börjar med 1-35 men siktar på nummervärden mellan 1-50. Spelare måste givetvis vara över 18 år men inkomst är inte ett krav. Alla över 65 år får en rad gratis om de spelar i pensionärsrabatt.

Uppgiftskrav

Oavsett vilket uppdrag gruppen väljer ovan ska samtliga delar finnas med och projektet ska knyta samman kursens olika moment. Det är nu ni visar att ni kan tillämpa de kunskaper och erfarenheter ni samlat på er under kursens gång. Notera att hela projektarbetet (veckorna nedan) är en **Agil iterativ process!**

Baserat på vecka 1, Problemlösning – analysera ett problem

Besvara de 5 Ws: Who, What, When, Where, Why: https://en.wikipedia.org/wiki/Five_Ws. Analysera ert uppdrag och fånga krav. Skapa en kravlista och UML use-case diagram. Kärna ut user stories. Hittar ni funktionella och icke funktionella krav? Lista dem på ett strukturerat och tydligt sätt. Vad hittar ni för objekt/klasser i ert uppdrag? Bryt ned problemet.

Baserat på vecka 2, APIE

Rent praktiskt, hur tänker ni designa programmet? Vilka relationer finns det mellan klasserna ni hittat? Förklara hur ni arbetar med Abstraktion, Polymorphism, Inheritance och Encapsulation. Varför är dessa viktiga aspekter? Hur passar APIE in med senare SOLID-principer? Presentera interface och klassers relationer med hjälp av UML-klassdiagram.

Baserat på vecka 3, SOLID

SOLID principer. Kunden behöver ett program som följer "best practice". De vill ha ett system som är skalbart och som deras egna IT-avdelning ska kunna förvalta. Vilka principer är lämpliga för uppdraget? Vad rekommenderar ni? Varför det? Hur ska det tillämpas i praktiken?

Baserat på vecka 4, Datastrukturer

Valet av datastrukturer är ett viktigt beslut. Därför förklarar ni hur ni resonerat tydligt inför kunden. Det finns kanske flera alternativ som skulle kunna passa som medför olika följder, bra eller dåliga. Ni kan låta kunden vara delaktig, men ge dem då ert underlag för att de ska kunna medverka i beslutet.

Baserat på vecka 5, Design Patterns

Kunden kanske blir osäker på all information de får. De känner sig överväldigade av all källkod de sett. Betrygga dem om att lösningen är genomtänkt genom att bl.a. sätta den i ett eller flera design patterns perspektiv. Förklara att er lösning bygger på välkända och tidigare utprovade metoder. Ge exempel på fördelar och eventuella nackdelar.

Baserat på vecka 6, Algoritmer

Bara ordet algoritm känns främmande för er kund. Lugna dem genom att definiera era valda algoritmer i er problemkontext. Beroende på vilken datastruktur ni kommit överens om kan ni ge exempel på hur till exempel en sökning eller sortering skulle kunna gå till.

Demonstrationer

Demonstrationer av programvaran vid redovisningen ska ske på *färdigskriven* kod. Ni ska alltså inte koda live. Gruppens implementation ska kunna byggas (kompileras) utan problem vid demonstrationerna.

För betyget G

Gruppen visar sitt lösningsförslag på mjukvara samt minst två (2) demonstrationer av kod relaterat till den projektuppgift man valt. Projektarbetet behöver inte resultera i en "färdig" mjukvara. Man behöver inte programmera hela lösningen och skapa ett UI/GUI.

Det som krävs för en lyckad demonstration för G är

Demo 1 – APIE demonstration

- minst ett (1) interface
- minst tre (3) klasser med "några" funktioner implementerade
- minst två (2) implementationer (arv/implements) av ett (eller flera) interface
- minst två (2) tydliga val av *inheritance*, eller *composition* som bättre alternativ

Demo 2 – Datastrukturer demonstration

Demonstrera minst två (2) CRUD-operationer med objekt skapade via tidigare demonstrerade klasser. En delmängd av mjukvaran där gruppen visar en implementation av någon central funktion i programmet via datastrukturer. Exempelvis ett konsolprogram där ni lägger till data, söker, sorterar, raderar osv. Välj det som känns mest relevant för ert valda projekt. Ni måste kunna motivera varför en viss datastruktur föredras före en annan, för- och nackdelar, i det aktuella sammanhanget det handlar om.

För betyget VG

Allt för G. Demonstrera en sammanhängande och fungerande mjukvara som en Stakeholder (Intressent) skulle kunna provköra eller gruppen kan "simulera/emulera" användningen/funktionen av enligt de krav som ställts på den projektuppgift man valt.

Beroende på projektuppgift kan ett enklare UI/GUI behövas för att demonstrera mjukvaran.

Bedömning

Nedanstående punkter kontrolleras. Saknas något är det inte godkänt

- ☐ Hur ni arbetat och presenterar ert projekt (alla ska bidra)
- ☐ Kravlista – funktionella och icke funktionella krav eller enligt FURPS
 - GDPR: <https://sv.wikipedia.org/wiki/Dataskyddsf%C3%B6rordningen>
 - CRA: <https://digital-strategy.ec.europa.eu/en/library/cyber-resilience-act-factsheet>
 - NIS-direktivet: <https://sv.wikipedia.org/wiki/NIS-direktivet> och uppföljaren NIS2-direktivet: <https://www.msb.se/sv/amnesomraden/informationssakerhet-cybersakerhet-och-sakra-kommunikationer/krav-och-regler-inom-informationssakerhet-och-cybersakerhet/nis-direktivet/det-har-ar-nis2-direktivet/>
- ☐ Use-case diagram
- ☐ User stories
- ☐ UML-klassdiagram med relationer
- ☐ SOLID-principer
- ☐ Design patterns
- ☐ Datastrukturer – minst två alternativ så kunden har något att välja mellan
- ☐ Algoritmer
- ☐ Mjukvarans kvalitet relaterat till den ställda projektuppgiften. Software quality definitions
 - The degree to which a component, system or process meets specified requirements and/or user/customer needs and expectations.
 - The totality of functionality and features of a software product that bear on its ability to satisfy stated or implied needs.

Följande demonstrationer visas för G och VG

- ☐ Se rubriken demonstrationer

Avslutning

Stort lycka till önskar Hans x 2